# LABORATORIO 08

## IMPLEMENTACIÓN DE TESTS

API REST de productos y categorías con Spring Boot

| | |
|---|---|
| **Estudiante** | Jesus Castillo Sumire |
| **Curso** | Desarrollo de Aplicaciones Web |
| **Fecha** | 19 de octubre de 2026 |
| **Proyecto** | API de Productos — laboratorio independiente |

## Objetivo

Verificar el funcionamiento de la API de productos mediante pruebas automatizadas y ampliar el proyecto con el CRUD de categorías. La solución cubre resultados correctos, datos inválidos, recursos inexistentes y duplicados, con persistencia de prueba aislada de MySQL.

## Cómo se trabajó

Se partió de una copia independiente del proyecto API de productos de la Semana 05. Se agregaron pruebas unitarias para los servicios, pruebas HTTP para ambos controladores y una funcionalidad de categorías en capas. H2 se usa durante las pruebas; XAMPP/MySQL se conserva para la ejecución local.

## Herramientas

| Tecnología | Uso |
|---|---|
| Java 17, Spring Boot 3.5.6 | Aplicación REST, JPA, validación y configuración por capas. |
| JUnit 5, MockMvc, H2 | Pruebas unitarias, de integración y base temporal. |
| Maven Wrapper, VS Code | Compilación, ejecución y edición. |
| XAMPP/MySQL, Postman | Persistencia local y pruebas manuales de endpoints. |

## Resumen del procedimiento

Se completó la actualización de productos, se creó el CRUD de categorías y se verificaron operaciones exitosas, validaciones y errores. Maven ejecutó 28 pruebas: 0 fallidas, 0 errores y 0 omitidas.

# 1. Arquitectura, base de datos y flujo

La solución separa las solicitudes HTTP, las reglas del negocio, la persistencia y el modelo de datos. Los DTO validan la entrada y el manejador global transforma errores en respuestas HTTP.

## Base de datos

La aplicación se conecta a MySQL/MariaDB de XAMPP con la base **laboratorio_productos**. La URL usa `createDatabaseIfNotExist=true` y JPA genera o actualiza sus tablas. Las pruebas usan H2 en memoria y no alteran datos de XAMPP.

```
CREATE DATABASE IF NOT EXISTS laboratorio_productos
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

## Modelo implementado

| Entidad | Campos / función | Relación |
|---|---|---|
| Producto | id, nombre, precio, stock, categoria (texto) | Conserva el modelo de Semana 05; sin FK nueva. |
| Categoria | id, nombre único, descripción | CRUD independiente. |
| AuditoriaLog | acción, método, fecha, detalle | Mantenida del proyecto base. |

Para iniciar: iniciar MySQL en XAMPP y ejecutar `./mvnw.cmd spring-boot:run`. La API corre en `http://localhost:8080`; Apache no es necesario. Las pruebas usan `./mvnw.cmd clean test` y no requieren XAMPP.

## 2. Cambios y endpoints

El servicio de productos actualiza por identificador; si el registro no existe, el controlador devuelve 404.

```
public Producto actualizar(Long id, Producto cambios) {
return repository.findById(id).map(producto -> {
producto.setNombre(cambios.getNombre());
producto.setPrecio(cambios.getPrecio());
producto.setStock(cambios.getStock());
producto.setCategoria(cambios.getCategoria());
return repository.save(producto);
}).orElse(null);
}
```

### El DTO de categoría valida campos y longitud

```
@NotBlank(message = "El nombre de la categoría es obligatorio")
@Size(max = 80)
private String nombre;

@Size(max = 255)
private String descripcion;
```

### Creación de categoría responde 201 y ubicación del nuevo recurso

```
@PostMapping
public ResponseEntity<CategoriaDTO> crear(
@Valid @RequestBody CategoriaDTO dto, UriComponentsBuilder uriBuilder) {
CategoriaDTO creada = service.crear(dto);
URI location = uriBuilder.path("/api/categorias/{id}")
.buildAndExpand(creada.getId()).toUri();
return ResponseEntity.created(location).body(creada);
}
```

### Rutas principales

| Método | Ruta | Resultado |
|---|---|---|
| GET | /api/productos[/{id}] | Listado y consulta. |
| POST / PUT / DELETE | /api/productos[/{id}] | CRUD de productos; PUT/DELETE inexistente: 404. |
| GET | /api/categorias[/{id}] | Listado y consulta de categorías. |
| POST / PUT / DELETE | /api/categorias[/{id}] | CRUD; 201 + Location al crear, 204 al borrar. |

### Ejemplo de solicitud

```
POST http://localhost:8080/api/categorias
Content-Type: application/json

{"nombre":"Accesorios","descripcion":"Periféricos y complementos"}
```

## 3. Evidencia de pruebas

MockMvc envía solicitudes a la aplicación Spring y comprueba los códigos de estado, el JSON de respuesta y los datos guardados en H2. Las pruebas unitarias revisan los servicios de forma aislada.

| Clase | Escenarios | Aprobadas |
|---|---|---|
| ProductoServiceTest | Reglas CRUD, consulta, actualización y casos no encontrados. | 7 |
| ProductoControllerTest | HTTP, validación, creación, actualización y eliminación. | 7 |
| CategoriaServiceTest | CRUD, duplicados y recurso inexistente. | 7 |
| CategoriaControllerTest | HTTP, validación, Location, 404 y 409. | 7 |

### Resultado comprobado

```
[INFO] CategoriaControllerTest: 7 pruebas, 0 fallidas, 0 errores
[INFO] ProductoControllerTest: 7 pruebas, 0 fallidas, 0 errores
[INFO] CategoriaServiceTest: 7 pruebas, 0 fallidas, 0 errores
[INFO] ProductoServiceTest: 7 pruebas, 0 fallidas, 0 errores
[INFO] Tests run: 28, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

### Comando reproducible

```
./mvnw.cmd clean test
```

| Estado | Caso verificado |
|---|---|
| 201 | Categoría creada y cabecera Location. |
| 400 | Campos obligatorios o longitud inválida. |
| 404 | Identificador inexistente. |
| 409 | Nombre de categoría duplicado sin distinguir mayúsculas. |
| 204 | Categoría eliminada. |

## 4. Observaciones y conclusiones

### Observaciones

**1.** Las pruebas unitarias permiten revisar la lógica de cada servicio sin iniciar MySQL ni depender de los datos de XAMPP.

**2.** MockMvc prueba las rutas HTTP y las respuestas de los controladores dentro del contexto Spring.

**3.** H2 se crea para las pruebas y se destruye al terminar; los datos de laboratorio_productos se mantienen aislados.

**4.** Categoría tiene entidad y CRUD propios; el campo categoría de producto se mantiene como texto para conservar compatibilidad con Semana 05.

**5.** La ejecución completa terminó con 28 pruebas aprobadas, incluyendo entradas inválidas, duplicados y consultas sin resultado.

### Conclusiones

**1.** Las pruebas unitarias comprobaron las reglas de negocio antes de depender de una ejecución manual.

**2.** Las pruebas de integración confirmaron que las rutas conectan controlador, validación, servicio y persistencia.

**3.** El CRUD de categorías amplía la API y mantiene separadas las responsabilidades de cada capa.

**4.** H2 facilita repetir pruebas sin modificar registros reales ni necesitar que MySQL esté encendido.

**5.** Maven terminó sin fallos; puedo repetir la verificación con ./mvnw.cmd clean test.

## ENTREGA DEL LABORATORIO 08

Proyecto, documentación técnica y pruebas automatizadas para productos y categorías.